SUPERMERCADO DB

Guía de Estudio — Backend

Node.js · Express 5 · MySQL2 · REST API · Preparación para Entrevista Técnica

📌 Este documento cubre desde los fundamentos de Node/Express hasta el análisis línea por línea de cada archivo del backend. Diseñado para que puedas defender tu prueba técnica con confianza total.

PARTE 1 · Fundamentos del Backend

1.1 ¿Qué es Node.js?

Node.js es un entorno de ejecución de JavaScript fuera del navegador. Está construido sobre el motor V8 de Chrome y permite ejecutar JS en el servidor.

Concepto	Descripción
`Runtime`	Node puede ejecutar JS en el servidor, no solo en el navegador.
`Non-blocking I/O`	Las operaciones lentas (base de datos, archivos) no bloquean el hilo principal. Node sigue atendiendo otras peticiones mientras espera.
`Single-threaded`	Node corre en un solo hilo, pero maneja concurrencia con el Event Loop.
`npm`	El gestor de paquetes de Node. Instala librerías como Express, mysql2, dotenv.
`ES Modules`	Este proyecto usa import/export (ESM) en vez de require/module.exports (CommonJS) porque package.json tiene "type": "module".

1.2 ¿Qué es Express?

Express es el framework web más popular para Node.js. Simplifica la creación de servidores HTTP y APIs REST al ofrecer un sistema de rutas y middlewares.

> 💡 Este proyecto usa Express 5 (^5.2.1) — la versión más reciente.
> Express 5 maneja errores async automáticamente, por lo que no necesitas wrappear con try/catch para que los errores no rompan el servidor (aunque en este proyecto igual se usa try/catch para dar mensajes claros).

API Express	Para qué sirve
`app.use()`	Registra un middleware global (JSON parser, CORS, etc.)
`app.get/post/put/delete()`	Define un endpoint para un método HTTP específico.
`router.get/post/...()`	Igual que app, pero en un Router modular separado.
`req`	Objeto Request: contiene params, body, query, headers de la petición.

`res`	Objeto Response: métodos para enviar la respuesta (json, status, send).
`next()`	Función para pasar al siguiente middleware en la cadena.

1.3 ¿Qué es una REST API?

REST (Representational State Transfer) es un estilo de arquitectura para APIs HTTP. Define convenciones sobre cómo usar los verbos HTTP y las URLs para operar sobre recursos.

Verbo HTTP	Endpoint	Acción	Respuesta típica
`GET`	`/productos`	Listar todos	200 + array JSON
`GET`	`/productos/:id`	Obtener uno por ID	200 + objeto JSON
`POST`	`/productos`	Crear nuevo	201 + objeto creado
`PUT`	`/productos/:id`	Actualizar por ID	200 + objeto actualizado
`DELETE`	`/productos/:id`	Eliminar por ID	200 + { message }
`GET`	`/inventario`	Existencias (via SP)	200 + array JSON
`POST`	`/inventario`	Nueva entrada	201 + { message }
`PUT`	`/inventario/:id`	Actualizar por ID	200 + objeto
`PUT`	`/inventario/codigo/:c`	Actualizar por código	200 + { inventario }
`DELETE`	`/inventario/:id`	Eliminar entrada	200 + { message }

PARTE 2 · Estructura del Proyecto Backend

2.1 Árbol de archivos

```
backend-express/
│
├── package.json          ← dependencias, scripts, tipo de módulo
├── .env                  ← variables de entorno (NO se sube a git)
│
└── src/
    ├── server.js         ← punto de entrada: inicia el servidor HTTP
    ├── app.js            ← configura Express: middlewares y rutas
    │
    ├── config/
    │   └── db.js         ← pool de conexiones a MySQL
    │
    ├── routes/
    │   ├── productos.routes.js    ← define qué URL llama a qué controller
    │   └── inventario.routes.js
    │
    └── controllers/
        ├── productos.controllers.js   ← lógica de negocio y SQL
        └── inventario.controllers.js
```

2.2 Patrón de arquitectura: Routes → Controllers

El proyecto usa una separación en capas muy común en APIs Express:

Archivo/Capa	Responsabilidad
`server.js`	Arranca el servidor. No sabe nada de rutas ni de base de datos.
`app.js`	Configura Express: aplica middlewares globales y registra los routers.
`routes/*.js`	Mapea URLs + verbos HTTP a funciones controller. Solo enrutamiento, sin lógica.
`controllers/*.js`	Contiene la lógica real: valida datos, consulta MySQL, devuelve respuesta.
`config/db.js`	Crea y exporta el pool de conexiones. Un solo lugar para la config de DB.
`.env`	Almacena secretos (contraseñas, puertos) fuera del código fuente.

💡 ¿Por qué separar routes de controllers?
— Si mañana cambia la URL de un endpoint, solo tocas routes.js.

— Si cambia la lógica SQL, solo tocas controllers.js.
— Cada archivo tiene una única razón para cambiar (Principio de Responsabilidad Única).

PARTE 3 · Análisis de Cada Archivo

3.1 package.json

> *Define las dependencias del proyecto, los scripts de ejecución y el tipo de módulo.*

```
{
  "name": "backend-express",
  "type": "module",          // ← CLAVE: habilita import/export (ES Modules)
  "scripts": {
    "dev": "node --watch ./src/server.js"
    //       ^ --watch recarga automáticamente al guardar (como nodemon)
  },
  "dependencies": {
    "cors":    "^2.8.6",      // permite peticiones desde el frontend (otro
origen)
    "dotenv":  "^17.3.1",     // carga variables del archivo .env
    "express": "^5.2.1",      // framework web
    "mysql2":  "^3.18.0"      // driver MySQL con soporte de Promises
  }
}
```

- **"type": "module"**: sin esto, Node usaría CommonJS (require). Con esto, puedes usar import/export en todos los archivos .js.
- **--watch**: flag nativo de Node 18+. Reinicia el servidor cada vez que guardas un archivo. Reemplaza a nodemon sin instalar nada extra.
- **mysql2 vs mysql**: mysql2 es la versión moderna con soporte nativo de Promises y async/await. La librería mysql original no tiene soporte de Promises.

3.2 .env (archivo que debes crear manualmente)

> *Almacena configuración sensible fuera del código. NUNCA se sube a GitHub.*

```
# Archivo .env en la raíz de backend-express/
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_contraseña
DB_NAME=supermercado_db
PORT=9874
```

⚠️ Agrega .env a tu .gitignore para que no se suba al repositorio:
echo '.env' >> .gitignore

Si alguien clona tu repositorio, debe crear su propio .env con sus credenciales.

3.3 src/server.js

> *Punto de entrada del backend. Su única responsabilidad es iniciar el servidor HTTP.*

```
import dotenv from 'dotenv';
import app from './app.js';

dotenv.config();   // carga las variables de .env en process.env

const port = process.env.PORT || 3000;
//                              ^ fallback si no hay .env

app.listen(port, () => {
  console.log(`Servidor corriendo en http://localhost:${port}`);
});
```

- **dotenv.config()**: lee el archivo .env y pone cada variable en process.env. Debe llamarse antes de cualquier código que use process.env.
- **process.env.PORT**: variable de entorno. En producción (Railway, Render, Heroku) el hosting define PORT automáticamente.
- **app.listen()**: arranca el servidor HTTP. A partir de aquí Express empieza a escuchar peticiones en el puerto indicado.
- server.js NO importa rutas ni controllers directamente — esa es responsabilidad de app.js. Separación limpia de responsabilidades.

3.4 src/app.js

> *Configura la instancia de Express: middlewares globales y registro de rutas.*

```
import express from 'express';
import productosRoutes  from './routes/productos.routes.js';
import inventarioRoutes from './routes/inventario.routes.js';
import cors from 'cors';

const app = express();   // crea la aplicación Express

app.use(express.json()); // middleware: parsea body JSON automáticamente
app.use(cors());         // middleware: añade headers CORS a todas las respuestas

app.use('/productos',  productosRoutes);   // monta el router en /productos
app.use('/inventario', inventarioRoutes);  // monta el router en /inventario

export default app;
```

¿Qué es un Middleware?

Un middleware es una función que se ejecuta entre que llega la petición y se envía la respuesta. Express procesa los middlewares en orden, de arriba a abajo.

```
// Flujo de una petición POST /productos:

Petición llega
    ↓
express.json()   → parsea el body JSON → popula req.body
    ↓
cors()           → añade Access-Control-Allow-Origin al response
    ↓
productosRoutes  → busca ruta que coincida con POST /
    ↓
createProducto() → ejecuta la lógica del controller
    ↓
res.json(...)    → envía la respuesta al cliente
```

- **express.json()**: sin este middleware, req.body sería undefined. Es esencial para recibir datos desde el frontend.
- **cors()**: sin este middleware, el navegador bloquearía las peticiones desde localhost:5173 (frontend) a localhost:9874 (backend) por la política de Same-Origin.
- **app.use('/productos', router)**: todas las rutas definidas en el router se prefijarán con /productos. La ruta '/' dentro del router se convierte en '/productos/'.

3.5 src/config/db.js

> *Crea y exporta el pool de conexiones a MySQL. Se importa en los controllers.*

```
import mysql from 'mysql2/promise';
//                        ^ /promise: habilita async/await nativamente
import dotenv from 'dotenv';
dotenv.config();

export const pool = mysql.createPool({
  host:     process.env.DB_HOST,      // 'localhost'
  user:     process.env.DB_USER,      // 'root'
  password: process.env.DB_PASSWORD,  // tu contraseña
  database: process.env.DB_NAME,      // 'supermercado_db'
});
```

Pool vs Conexión única

Un pool mantiene un conjunto de conexiones reutilizables abiertas en lugar de abrir y cerrar una conexión en cada petición.

Concepto	Descripción
`Conexión única`	Abre → consulta → cierra en cada request. Lento, no escala.
`Pool de conexiones`	Mantiene N conexiones listas. Cada request toma una libre, la usa y la devuelve al pool.
`mysql2/promise`	Versión de mysql2 que retorna Promises en vez de usar callbacks. Esencial para async/await.
`pool.query()`	Toma una conexión del pool, ejecuta la query, devuelve la conexión. Todo automático.

PARTE 4 · Rutas (routes/)

4.1 productos.routes.js

> *Define los 5 endpoints del CRUD de productos. Solo enrutamiento, sin lógica.*

```
import { Router } from 'express';
import { getProductos, getProductoById, createProducto,
         updateProducto, deleteProducto } from
'../controllers/productos.controllers.js';

const router = Router();  // mini-aplicación Express reutilizable

router.get('/',    getProductos);      // GET  /productos
router.get('/:id', getProductoById);   // GET  /productos/42
router.post('/',   createProducto);    // POST /productos
router.put('/:id', updateProducto);    // PUT  /productos/42
router.delete('/:id', deleteProducto); // DELETE /productos/42

export default router;
```

- **Router()**: crea un router modular. Es como una mini-app de Express que luego se monta en app.js con app.use('/productos', router).
- **:id en la ruta**: define un parámetro dinámico. Express lo captura en req.params.id automáticamente.
- El orden de las rutas importa: Express las evalúa de arriba a abajo y usa la primera que coincide.

4.2 inventario.routes.js

> *5 endpoints de inventario. Atención especial al orden de las rutas PUT.*

```
router.get('/',           getInventario);
router.post('/',          createInventario);
router.put('/codigo/:codigo', updateInventarioByCodigo);  // ← PRIMERO
router.put('/:id',        updateInventario);              // ← DESPUÉS
router.delete('/:id',     deleteInventario);
```

⚠️ Orden crítico de las rutas PUT:
Si pusieras '/:id' ANTES que '/codigo/:codigo', una petición a PUT /inventario/codigo/PRD-0001 matchearía '/:id' con id='codigo' en vez de la ruta correcta. Las rutas específicas van siempre ANTES que las rutas genéricas con parámetros.

PARTE 5 · Controllers — Productos

5.1 getProductos (GET /productos)

> *Retorna todos los productos de la tabla.*

```
export const getProductos = async (req, res) => {
  try {
    const [result] = await pool.query('SELECT * FROM productos');
    //     ^ desestructuración: pool.query retorna [rows, fields]
    //       solo necesitamos rows
    res.json(result);   // 200 OK + array de productos
  }
  catch (error) {
    console.error('Error al obtener productos:', error);
    res.status(500).json({ error: 'Error al obtener productos' });
  }
};
```

- **async/await**: pool.query() retorna una Promise. await pausa hasta que MySQL responde.
- **const [result]**: mysql2 retorna [rows, fieldMetadata]. Con desestructuración tomamos solo el primer elemento (las filas).
- **res.json()**: serializa el objeto/array a JSON y lo envía con Content-Type: application/json. Status 200 implícito.
- **try/catch**: captura errores de red o SQL. Sin esto, un error no capturado en Express 4 colgaría el servidor (en Express 5 se maneja automáticamente, pero es buena práctica igual).

5.2 getProductoById (GET /productos/:id)

```
export const getProductoById = async (req, res) => {
  const id = parseInt(req.params.id);
  //                  ^ string de la URL → número entero

  try {
    const [result] = await pool.query(
      'SELECT * FROM productos WHERE id_producto = ?', [id]
    //                                               ^ placeholder
    //                                                  ^ array de valores
    );

    if (result.length === 0) {
      return res.status(404).json({ error: 'Producto no encontrado' });
    }

    res.json(result[0]);  // retorna el primer (y único) objeto
```

```
  }
  catch (error) { ... }
};
```

🔒 Placeholders (?) y SQL Injection:
NUNCA concatenes variables directamente en la query:
❌ 'SELECT * FROM productos WHERE id = ' + id
✅ 'SELECT * FROM productos WHERE id = ?', [id]

mysql2 sanitiza automáticamente los valores cuando usas ?.
La concatenación directa permite ataques de SQL Injection.

5.3 createProducto (POST /productos)

```
export const createProducto = async (req, res) => {
  // Desestructura del body (req.body fue parseado por express.json())
  const { codigo_producto, nombre_producto, categoria,
          sub_categoria, marca } = req.body;

  try {
    const [result] = await pool.query(
      'INSERT INTO productos (codigo_producto, nombre_producto,
       categoria, sub_categoria, marca) VALUES (?, ?, ?, ?, ?)',
      [codigo_producto, nombre_producto, categoria, sub_categoria, marca]
    );

    res.status(201).json({
      id: result.insertId,   // ID auto-generado por MySQL
      codigo_producto, nombre_producto, categoria, sub_categoria, marca
    });
  }
  catch (error) { ... }
};
```

- **req.body**: contiene los datos enviados en el cuerpo de la petición POST. Disponible porque app.use(express.json()) lo parseó.
- **result.insertId**: MySQL retorna el ID auto-incrementado del registro insertado. Útil para devolver el objeto completo al frontend.
- **status(201)**: HTTP 201 Created indica que el recurso fue creado exitosamente. Es más semántico que 200 para operaciones POST.

5.4 updateProducto (PUT /productos/:id)

```
export const updateProducto = async (req, res) => {
  const id = parseInt(req.params.id);
  const { codigo_producto, nombre_producto, categoria,
          sub_categoria, marca } = req.body;

  try {
    const [result] = await pool.query(
      'UPDATE productos SET codigo_producto=?, nombre_producto=?,
       categoria=?, sub_categoria=?, marca=? WHERE id_producto=?',
      [codigo_producto, nombre_producto, categoria, sub_categoria, marca, id]
    );

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Producto no encontrado' });
    }

    res.status(201).json({ id, codigo_producto, ... });
  }
  catch (error) { ... }
};
```

- **result.affectedRows**: número de filas modificadas. Si es 0, el ID no existía en la base de datos → retornamos 404.

5.5 deleteProducto (DELETE /productos/:id)

```
export const deleteProducto = async (req, res) => {
  const id = parseInt(req.params.id);

  try {
    const [result] = await pool.query(
      'DELETE FROM productos WHERE id_producto = ?', [id]
    );

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Producto no encontrado' });
    }

    res.json({ message: 'Producto eliminado correctamente' });
  }
  catch (error) { ... }
};
```

💡 La tabla inventario tiene ON DELETE CASCADE en la FK fk_producto.

Esto significa que al eliminar un producto, MySQL elimina automáticamente todas sus entradas de inventario asociadas. No necesitas hacer DELETE manual en inventario desde el código.

PARTE 6 · Controllers — Inventario

6.1 getInventario (GET /inventario)

> *Llama al Stored Procedure sp_inventario_existencias() para obtener existencias totales.*

```
export const getInventario = async (req, res) => {
  try {
    const [result] = await pool.query('CALL sp_inventario_existencias()');
    //                                ^ llama al Stored Procedure
    res.json(result[0]);
    //            ^ CALL retorna [[rows], okPacket]. result[0] son las filas.
  }
  catch (error) { ... }
};
```

💡 ¿Por qué result[0] y no result?
pool.query('CALL sp...') retorna [[filas_del_SP], OkPacket]
Es decir, result es un array de dos elementos.
result[0] son las filas de datos que retorna el SP.
result[1] es metadata de la ejecución (no nos interesa).

6.2 createInventario (POST /inventario)

> *Inserta una nueva entrada de inventario buscando el id_producto a partir del código.*

```
export const createInventario = async (req, res) => {
  const { codigo_producto, fecha_ingreso, cantidad,
          costo, fecha_vencimiento } = req.body;

  try {
    const [result] = await pool.query(
      `INSERT INTO inventario (fecha_ingreso, cantidad, costo,
                               fecha_vencimiento, id_producto)
       SELECT ?, ?, ?, ?, p.id_producto
       FROM productos p
       WHERE p.codigo_producto = ?`,
      [fecha_ingreso, parseInt(cantidad), parseFloat(costo),
       fecha_vencimiento || null, codigo_producto]
    );

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Producto no encontrado' });
    }
```

```
    res.status(201).json({ message: 'Producto añadido' });
  }
  catch (error) { ... }
};
```

Técnica INSERT ... SELECT

En vez de hacer dos queries (primero GET para obtener el id_producto, luego INSERT), esta query hace todo en uno:

- **INSERT INTO inventario ... SELECT**: inserta el resultado de un SELECT directamente.
- Si el código no existe, el SELECT retorna 0 filas → affectedRows = 0 → retornamos 404.
- **fecha_vencimiento || null**: si el frontend envía string vacío '', se convierte a null para que MySQL no falle (el campo permite NULL).
- **parseInt / parseFloat**: aunque el frontend envíe strings del input HTML, los convertimos a número antes de guardar en MySQL.

6.3 updateInventarioByCodigo (PUT /inventario/codigo/:codigo)

> *El endpoint más complejo: busca la última entrada de un producto por código y la actualiza.*

```
export const updateInventarioByCodigo = async (req, res) => {
  const codigo = req.params.codigo;
  const { fecha_ingreso, cantidad, costo, fecha_vencimiento } = req.body;

  try {
    // PASO 1: Buscar el id de la última entrada de ese producto
    const [targetRows] = await pool.query(
      `SELECT i.id_inventario
       FROM inventario i
       JOIN productos p ON p.id_producto = i.id_producto
       WHERE p.codigo_producto = ?
       ORDER BY i.id_inventario DESC
       LIMIT 1`,
      [codigo]
    );

    if (targetRows.length === 0) {
      return res.status(404).json({ error: 'No se encontró entrada' });
    }

    const targetId = targetRows[0].id_inventario;

    // PASO 2: Actualizar esa entrada específica
    await pool.query(
      'UPDATE inventario SET fecha_ingreso=?, cantidad=?,
       costo=?, fecha_vencimiento=? WHERE id_inventario=?',
      [fecha_ingreso, parseInt(cantidad), parseFloat(costo),
```

```
        fecha_vencimiento || null, targetId]
    );

    // PASO 3: Retornar el registro actualizado (con JOIN para incluir código)
    const [rows] = await pool.query(
      `SELECT i.id_inventario, p.codigo_producto, i.fecha_ingreso,
              i.cantidad, i.costo, i.fecha_vencimiento
       FROM inventario i
       JOIN productos p ON p.id_producto = i.id_producto
       WHERE i.id_inventario = ?`,
      [targetId]
    );

    res.json({ message: 'Entrada actualizada', inventario: rows[0] });
  }
  catch (error) { ... }
};
```

Este controller usa un patrón de 3 queries secuenciales:

1. SELECT para encontrar el ID de la última entrada por código.
2. UPDATE para modificar esa entrada específica.
3. SELECT final para retornar los datos actualizados con JOIN (incluye el código del producto).

> 💡 ¿Por qué 3 queries en vez de hacer UPDATE ... JOIN directamente?
> MySQL sí permite UPDATE con JOIN, pero separarlo en pasos hace el código más legible y más fácil de depurar. Si algo falla, sabes exactamente en cuál de los 3 pasos ocurrió.

PARTE 7 · La Base de Datos (pt-query.sql)

7.1 Estructura de tablas

```
CREATE TABLE productos (
  id_producto     INT AUTO_INCREMENT PRIMARY KEY NOT NULL,
  codigo_producto VARCHAR(50)  NOT NULL UNIQUE,   -- no permite duplicados
  nombre_producto VARCHAR(100) NOT NULL,
  categoria       VARCHAR(100) NOT NULL,
  sub_categoria   VARCHAR(100) NOT NULL,
  marca           VARCHAR(100) NOT NULL
);

CREATE TABLE inventario (
  id_inventario    INT AUTO_INCREMENT PRIMARY KEY NOT NULL,
  fecha_ingreso    DATE         NOT NULL,
  cantidad         INT          NOT NULL,
  costo            DECIMAL(10,2) NOT NULL,   -- 10 dígitos, 2 decimales
  fecha_vencimiento DATE,                    -- permite NULL (opcional)

  id_producto INT,
  CONSTRAINT fk_producto FOREIGN KEY (id_producto)
    REFERENCES productos(id_producto)
    ON DELETE CASCADE    -- eliminar producto = eliminar su inventario
    ON UPDATE CASCADE    -- cambiar id_producto = actualiza la FK
);
```

Concepto SQL	Descripción
`AUTO_INCREMENT`	MySQL asigna automáticamente el siguiente número. No se envía en el INSERT.
`UNIQUE`	El campo no puede tener valores repetidos. codigo_producto debe ser único.
`DECIMAL(10,2)`	Número con hasta 10 dígitos en total y 2 decimales. Ideal para precios/costos.
`DATE`	Solo fecha (YYYY-MM-DD). Sin hora.
`FK + ON DELETE CASCADE`	Si eliminas un producto, MySQL borra automáticamente sus entradas de inventario.
`ON UPDATE CASCADE`	Si cambiara el id_producto (raro), la FK se actualiza automáticamente.

7.2 Stored Procedure sp_inventario_existencias

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_inventario_existencias()
BEGIN
  SELECT
    p.codigo_producto,
    p.nombre_producto,
    SUM(i.cantidad) AS existencia_total   -- suma de todas las entradas
  FROM productos p
  INNER JOIN inventario i
    ON i.id_producto = p.id_producto      -- une por la FK
  GROUP BY p.id_producto, p.codigo_producto, p.nombre_producto
  HAVING SUM(i.cantidad) > 0              -- solo productos con stock
  ORDER BY p.codigo_producto;
END $$

DELIMITER ;
```

Concepto	Descripción
`DELIMITER $$`	MySQL usa ; para terminar statements. Cambiamos el delimitador a $$ para que el ; dentro del SP no lo interrumpa.
`INNER JOIN`	Solo retorna filas que tienen coincidencia en AMBAS tablas. Productos sin inventario no aparecen.
`GROUP BY`	Agrupa todas las entradas de inventario del mismo producto en una sola fila.
`SUM(i.cantidad)`	Suma las cantidades de todas las entradas de ese producto.
`HAVING`	Como WHERE pero para resultados agrupados (GROUP BY). Filtra grupos, no filas individuales.
`CALL sp_...()`	Así se llama un SP desde SQL o desde Node: pool.query('CALL sp_inventario_existencias()')

PARTE 8 · Flujos Completos — Request/Response

8.1 GET /productos — Listar todos

```
Frontend:  fetch('http://localhost:9874/productos')
           ↓
Express:   app.use('/productos', productosRoutes)  → match
           ↓
Router:    router.get('/', getProductos)  → match GET /
           ↓
Controller: pool.query('SELECT * FROM productos')
           ↓
MySQL:     retorna array de filas
           ↓
Controller: res.json(result)  → serializa a JSON
           ↓
Frontend:  const productos = await response.json()
```

8.2 POST /productos — Crear

```
Frontend:  fetch('/productos', {
             method: 'POST',
             headers: { 'Content-Type': 'application/json' },
             body: JSON.stringify({ codigo_producto: 'PRD-006', ... })
           })
           ↓
Middleware: express.json() → parsea body → req.body = { codigo_producto: ... }
           ↓
Router:    router.post('/', createProducto)
           ↓
Controller: INSERT INTO productos ... VALUES (?, ?, ...)
           ↓
MySQL:     result.insertId = 6  (nuevo ID auto-generado)
           ↓
Controller: res.status(201).json({ id: 6, codigo_producto: 'PRD-006', ... })
           ↓
Frontend:  respuesta 201 Created + objeto del producto creado
```

8.3 PUT /inventario/codigo/PRD-0001 — Editar por código

```
Frontend:  fetch('/inventario/codigo/PRD-0001', { method: 'PUT', body: ... })
           ↓
Router:    router.put('/codigo/:codigo', updateInventarioByCodigo)
           req.params.codigo = 'PRD-0001'
```

```
           ↓
Controller PASO 1: SELECT id_inventario WHERE codigo_producto = 'PRD-0001'
                   ORDER BY id_inventario DESC LIMIT 1
                   → targetId = 5
           ↓
Controller PASO 2: UPDATE inventario SET ... WHERE id_inventario = 5
           ↓
Controller PASO 3: SELECT ... JOIN ... WHERE id_inventario = 5
                   → datos actualizados con nombre del producto
           ↓
Controller: res.json({ message: '...', inventario: { ... } })
```

PARTE 9 · Cómo Ejecutar el Backend

9.1 Pasos de instalación y ejecución

4. Instalar MySQL en tu máquina (si no lo tienes) y crear la base de datos:

```
mysql -u root -p < pt-query.sql
# o abrir MySQL Workbench y ejecutar el script manualmente
```

5. Crear el archivo .env en la carpeta raíz del backend:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_contraseña_de_mysql
DB_NAME=supermercado_db
PORT=9874
```

6. Instalar dependencias:

```
npm install
```

7. Iniciar el servidor:

```
npm run dev
```

8. Verificar que funciona:

```
# En el navegador o con curl:
http://localhost:9874/productos
http://localhost:9874/inventario
```

9. Iniciar el frontend React (en otra terminal):

```
cd supermercado-vite
npm run dev
# Abre http://localhost:5173
```

9.2 Estructura de carpetas final del backend

```
backend-express/
├── .env                  ← crea este archivo manualmente
├── package.json
└── src/
```

```
    ├── server.js
    ├── app.js
    ├── config/
    │   └── db.js
    ├── routes/
    │   ├── productos.routes.js
    │   └── inventario.routes.js
    └── controllers/
        ├── productos.controllers.js
        └── inventario.controllers.js
```

PARTE 10 · Preguntas de Entrevista — Backend

¿Qué es un middleware en Express?

Una función que se ejecuta entre que llega la petición HTTP y que se envía la respuesta. Tiene acceso a req, res y next(). Ejemplos: express.json() parsea el body, cors() añade headers. Se registran con app.use() y se ejecutan en orden.

¿Qué es CORS y por qué es necesario?

Cross-Origin Resource Sharing. El navegador bloquea por defecto peticiones desde un origen (localhost:5173) a otro origen (localhost:9874). El middleware cors() añade el header Access-Control-Allow-Origin a las respuestas para decirle al navegador que el backend acepta esas peticiones.

¿Por qué se usa un pool de conexiones en vez de una conexión directa?

Abrir una conexión a MySQL tarda tiempo. Un pool mantiene N conexiones ya abiertas y listas. Cada petición toma una conexión libre del pool, la usa y la devuelve. Esto mejora enormemente el rendimiento bajo carga y evita el overhead de abrir/cerrar conexiones en cada request.

¿Qué es SQL Injection y cómo lo previene este proyecto?

Un ataque donde el usuario introduce SQL malicioso en un input para manipular la base de datos. Este proyecto usa placeholders (?) con mysql2: los valores se sanitizan automáticamente antes de ejecutarse. Nunca se concatena directamente el input del usuario en la query.

¿Para qué sirve dotenv?

Lee el archivo .env y carga las variables como process.env.NOMBRE_VARIABLE. Esto permite mantener datos sensibles (contraseñas, API keys) fuera del código fuente, con valores diferentes por entorno (desarrollo, producción).

¿Qué hace parseInt() en los controllers y por qué es necesario?

Los parámetros de la URL (req.params.id) son siempre strings. MySQL espera un número entero para el WHERE id = ?. parseInt() convierte '42' a 42. Sin esto, la query puede fallar o dar resultados inesperados.

¿Por qué el Stored Procedure usa DELIMITER $$?

MySQL usa el punto y coma ; para separar statements. Si no cambiáramos el delimitador, al definir el SP, el primer ; dentro del BEGIN/END terminaría el CREATE PROCEDURE prematuramente. Cambiamos a $$ para que el ; dentro del SP sea parte de su cuerpo.

¿Qué diferencia hay entre WHERE y HAVING?

WHERE filtra filas individuales ANTES de agrupar. HAVING filtra grupos DESPUÉS del GROUP BY. En el SP, HAVING SUM(i.cantidad) > 0 elimina productos que tengan existencia total de 0 o menos, lo cual no sería posible con WHERE porque SUM() aún no está calculado en esa etapa.

¿Por qué la ruta /codigo/:codigo va antes que /:id en inventario.routes.js?

Express evalúa las rutas en orden de arriba a abajo. Si /:id estuviera primero, la petición PUT /inventario/codigo/PRD-0001 coincidiría con /:id (con id='codigo') en vez de /codigo/:codigo. Las rutas específicas siempre deben declararse antes que las genéricas con parámetros dinámicos.

¿Qué retorna mysql2 al hacer pool.query()?

Retorna una Promise que resuelve a [rows, fieldMetadata]. Con desestructuración const [result] = await pool.query(...) tomamos solo el primer elemento (las filas). Para CALL sp_..., result es [[filas_sp], okPacket], por eso se accede a result[0] para obtener las filas del SP.

PARTE 11 · Glosario Backend

Término	Definición
`Node.js`	Entorno de ejecución de JavaScript en el servidor, basado en el motor V8 de Chrome.
`Express`	Framework web minimalista para Node.js. Simplifica rutas, middlewares y respuestas HTTP.
`REST API`	Arquitectura de API que usa verbos HTTP (GET/POST/PUT/DELETE) y URLs como recursos.
`Middleware`	Función que procesa la petición antes de llegar al handler final. app.use() la registra.
`Router`	Mini-app Express modular que agrupa rutas relacionadas. Se monta en app con app.use().
`Controller`	Función que contiene la lógica de negocio: valida datos, consulta DB, devuelve respuesta.
`req.params`	Parámetros dinámicos de la URL. En /productos/:id, req.params.id es el valor capturado.
`req.body`	Datos del cuerpo de la petición (POST/PUT). Disponible tras aplicar express.json().
`res.json()`	Envía respuesta JSON con Content-Type: application/json. Status 200 implícito.
`res.status()`	Encadena el código de status HTTP. res.status(404).json({error: '...'}).
`async/await`	Sintaxis para manejar Promises secuencialmente. await pausa hasta que la Promise resuelve.
`Pool`	Conjunto de conexiones MySQL reutilizables. Mejor rendimiento que abrir una conexión por request.
`mysql2/promise`	Versión de mysql2 con soporte nativo de Promises/async-await.
`Placeholder (?)`	Marcador en queries SQL que mysql2 reemplaza con valores sanitizados. Previene SQL Injection.
`affectedRows`	Propiedad del resultado de UPDATE/DELETE. Indica cuántas filas fueron modificadas.
`insertId`	Propiedad del resultado de INSERT. Contiene el ID auto-generado por MySQL.
`dotenv`	Librería que carga variables del archivo .env en process.env.
`CORS`	Política del navegador. El middleware cors() permite peticiones cross-origin desde el frontend.
`Stored Procedure`	Bloque de SQL guardado en MySQL con nombre. Se llama con CALL nombre().

`CASCADE`	Comportamiento de FK: ON DELETE CASCADE elimina registros hijos al eliminar el padre.
`DECIMAL(10,2)`	Tipo MySQL para números exactos con decimales. Ideal para precios. Evita errores de float.
`ES Modules`	Sistema moderno de módulos JS (import/export). Habilitado con "type": "module" en package.json.
`HTTP 200`	OK. La petición fue exitosa.
`HTTP 201`	Created. Se creó un nuevo recurso (respuesta ideal para POST/INSERT).
`HTTP 404`	Not Found. El recurso solicitado no existe.
`HTTP 500`	Internal Server Error. Error inesperado en el servidor.

¡Mucho éxito en tu entrevista!

Supermercado DB · Node.js + Express + MySQL · Guía de Estudio Backend